FOCUS REVIEW

LLM 重点学习手册

注意力机制与对齐损失 · 6 个重点专题
多注意力头 / 稀疏注意力 · SFT / DPO / GRPO / OPD

注意力机制 · 损失函数

2 个主题 · 6 个专题 · 生成于 2026-10-13

目录

注意力机制 • Attention

01 多头注意力　02 稀疏注意力

损失函数 • Loss

03 交叉熵 / 语言模型损失　04 DPO 损失　05 GRPO 损失　06 在线策略蒸馏

注意力机制 Attention

2 题

多头注意力

Multi-Head Attention

并行多头 → 拼接 → 输出投影

📌 原理与思想

核心概念

将输入投影到多个子空间，每个头独立计算注意力，最后拼接并通过线性层融合。相比单头注意力，多头机制允许模型同时关注不同位置的不同表示子空间，捕捉更丰富的语义关系。

核心思想

通过多个独立的注意力头，每个头学习不同的注意力模式（如语法关系、语义关系等）。最后通过输出投影融合所有头的信息，增强模型的表达能力。

中间那个注意力矩阵是 [B, H, S, S]，显存与头数成正比 —— 这正是 GQA、MQA 要砍掉的那一项；而输出投影 W_o 是唯一发生跨头交互的地方。

算法步骤

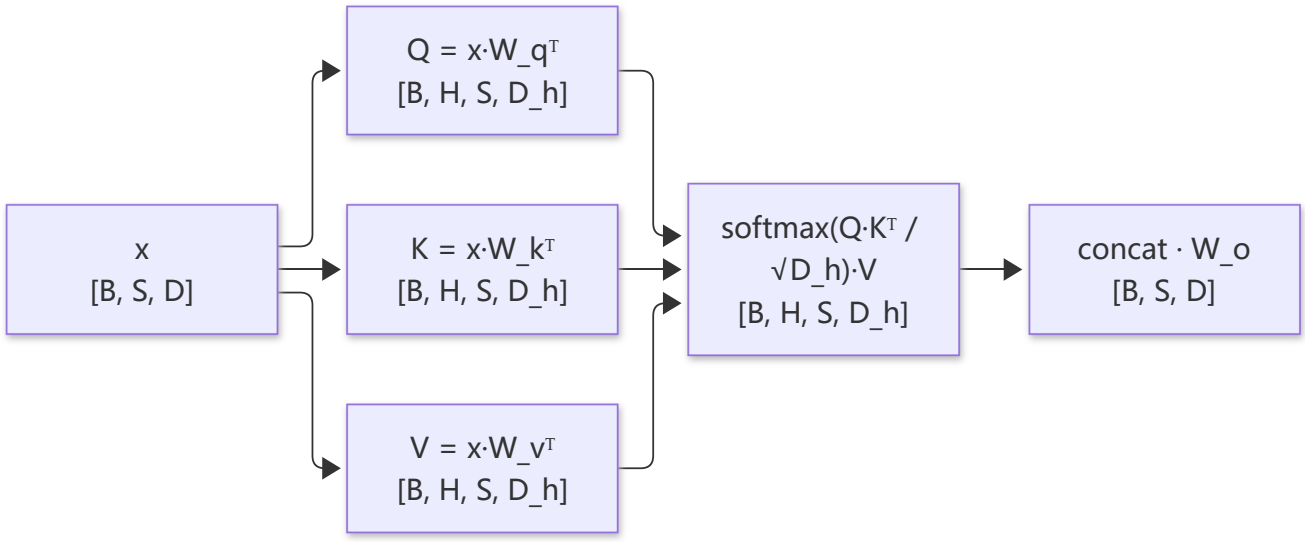
1. 线性投影：Q = x @ Wq, K = x @ Wk, V = x @ Wv
2. 分头：view(B, S, H, Dh).transpose(1, 2) → [B, H, S, Dh]
3. 对每个头计算缩放点积注意力
4. 合并多头：transpose(1, 2).contiguous().view(B, S, D)
5. 输出投影：output = concat_output @ Wo

📐 核心公式

$$\mathrm{MultiHead}(Q,K,V) = \mathrm{Concat}(\mathrm{head}_1,\ldots,\mathrm{head}_H)\,W_O$$

$$\mathrm{head}_i = \mathrm{Attention}\left(QW_i^Q,\ KW_i^K,\ VW_i^V\right), \qquad D_h = \frac{D}{H}$$

📊 张量流程图



💻 代码实现

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_heads):
        super().__init__()
        assert d_model % n_heads == 0
```

```
        self.h, self.dh = n_heads, d_model // n_heads

        self.wq = nn.Linear(d_model, d_model)
        self.wk = nn.Linear(d_model, d_model)
        self.wv = nn.Linear(d_model, d_model)
        self.wo = nn.Linear(d_model, d_model)

    def forward(self, xq, xk=None, mask=None):
        """
        xq: [batch, seq_len_q, d_model]
        xk: [batch, seq_len_k, d_model] (None for self-attention)
        mask: [batch, 1, seq_len_q, seq_len_k]
        """
        B = xq.size(0)
        xk = xk if xk is not None else xq  # self-attention

        # 线性投影
        q = self.wq(xq).view(B, -1, self.h, self.dh).transpose(1, 2)
        k = self.wk(xk).view(B, -1, self.h, self.dh).transpose(1, 2)
        v = self.wv(xk).view(B, -1, self.h, self.dh).transpose(1, 2)

        # 缩放点积注意力
        scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(self.dh)
        if mask is not None:
            scores = scores.masked_fill(mask == 0, -1e9)
        attn = F.softmax(scores, dim=-1)

        # 加权求和
        out = torch.matmul(attn, v)  # [B, H, S, Dh]

        # 合并多头
        out = out.transpose(1, 2).contiguous().view(B, -1, self.h * self.dh)
        return self.wo(out)

# 测试
if __name__ == "__main__":
    B, S, D, H = 2, 10, 64, 8
    x = torch.randn(B, S, D)
    mha = MultiHeadAttention(D, H)
    out = mha(x, x)  # self-attention
    print(f"Input shape: {x.shape}")   # [2, 10, 64]
    print(f"Output shape: {out.shape}") # [2, 10, 64]
```

🎯 面试要点

- **分头操作**: `view(B,S,H,Dh).transpose(1,2)` — 面试必写
- **合并操作**: `transpose(1,2).contiguous().view(B,S,D)`
- **自注意力**: Q=K=V=x
- **交叉注意力**: Q=x_dec, K=V=x_enc
- **head_dim = d_model / n_heads**，通常 64 或 128
- **参数量**: 与单头完全相同，4 × d_model² (Wq, Wk, Wv, Wo)，只多出一组中间张量（H·Dh = D）

稀疏注意力

Native Sparse Attention / DeepSeek Sparse Attention

长上下文下 O(S²) 不可行：压缩 + 选择 + 滑窗，或索引器 + top-k

📌 原理与思想

核心概念

长上下文注意力是 `O(S²)`，必须稀疏化。NSA 走三分支路线：压缩块注意力（粗粒度全局）、top-n 块选择（中粒度重要区域）、滑动窗口（局部精确），再用学到的门控加权求和；DSA 走另一条路，用一个极轻的 lightning indexer 给每个历史 token 打分，只对 top-k 个 token 做真正的注意力。两者都把复杂度降到 `O(S·n)` 或 `O(S·k)`，也都要求块对齐以适配硬件 —— 代价是块大小 `l` 成了超参，太小则选择本身变贵，太大则选得不够精细。

核心思想

注意力权重实际上是稀疏的 —— 绝大多数位置对当前 token 无关，稠密计算在浪费算力。难点在「怎么稀疏」：套固定模式（只看滑窗）会丢掉长程依赖，随机或启发式选择非连续，gather 会让 GPU 利用率崩掉。所以能用的方案必须同时满足三条：保住长程信息、选择是学出来的、粒度块对齐。

算法步骤

1. NSA：先把 `K/V` 按块均值池化成 `K_cmp`（块大小 `l`），用压缩后的表示做一次粗粒度注意力。
2. 同一份 `K_cmp` 的分数用来挑 top-n 个块，再取回这些块的**原始** `KV` 做精注意力；同时保留最近 `w` 个 token 的滑窗。
3. 三路输出按 `g = σ(wᵀ·[q_t ; ...])` 加权求和，门控是学出来的，模型自己决定何时依赖全局、何时只看局部。
4. DSA：`k_s = W^{K,l}·h_s` 是一个很轻的 key 投影（`d^I` 远小于 `d`），`I_{t,s} = Σ_j w_{t,j}·ReLU(q_{t,j}·k_s)` 给每个历史 token 打分，取 top-k 后再在这 `k` 个 token 上做 MLA 注意力。
5. Indexer 维度极低且能用 FP8 跑，所以「给所有历史 token 打分」这一步的代价可以忽略。

📐 核心公式

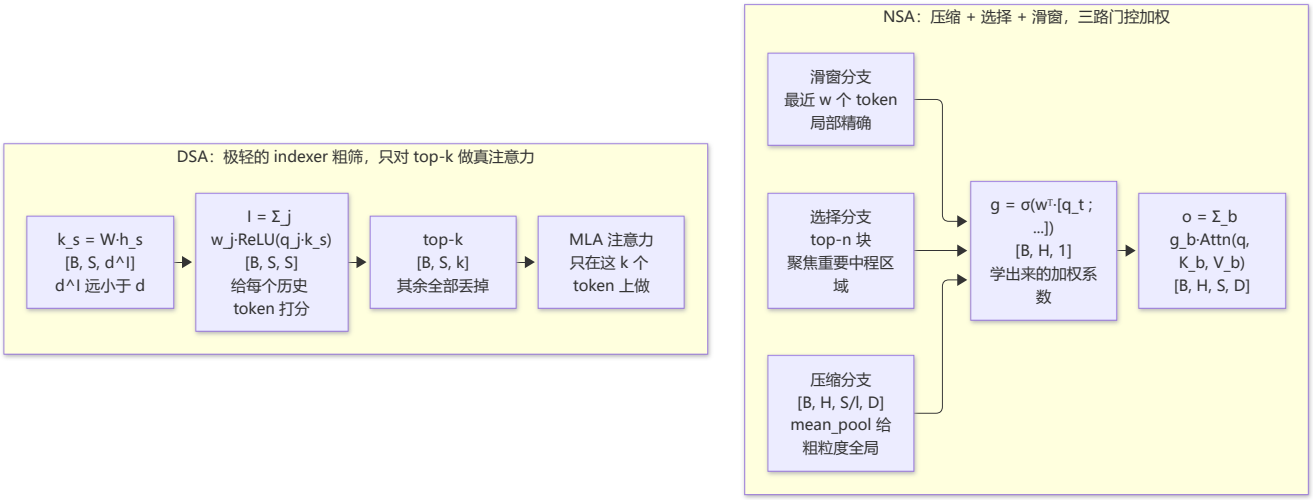
$$
\begin{aligned}
K_{\text{cmp},j} &= \text{mean}\big(K_{jl:(j+1)l}\big) \\
p_j &= \text{score}(q_t, K_{\text{cmp},j}), \qquad \mathcal{B}_{\text{slc}} = \text{top}_n(p) \\
\mathcal{B}_{\text{win}} &= \{t-w, \dots, t\}
\end{aligned}
$$

$$
o_t = \sum_{b \in \{\text{cmp},\text{slc},\text{win}\}} g_b \,\text{Attn}(q_t, K_b, V_b), \qquad g = \sigma\big(w^\top [q_t; \dots]\big)
$$

$$
k_s = W^{K,l} h_s, \qquad I_{t,s} = \sum_j w_{t,j} \text{ReLU}(q_{t,j} \cdot k_s), \qquad \mathcal{S}_t = \text{top}_k(I_{t,\cdot})
$$

$$
\mathcal{S}_t \subseteq \{s \le t\}, \qquad k = 2048, \qquad O(S^2) \to O(S\,k)
$$

📊 张量流程图



💻 代码实现

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
import math


def compress_blocks(x, block_size):
    """块内均值池化: [B,H,S,D] → [B,H,S//l,D]"""
    B, H, S, D = x.shape
    n_blocks = S // block_size
    x = x[:, :, :n_blocks * block_size].reshape(B, H, n_blocks, block_size, D)
    return x.mean(dim=3)                                    # [B, H, n_blocks, D]


def sdpa(q, k, v, scale):
    """标准缩放点积注意力（q 可以是多一个维度的 query 维）"""
    return F.softmax(q @ k.transpose(-1, -2) * scale, dim=-1) @ v


def nsa_attention(q, k, v, block_size=8, top_n=2, window=16):
    """
    NSA 三分支稀疏注意力
    q, k, v: [B, H, S, D]；返回 o: [B, H, S, D]
    注意：为简洁起见这里省略了因果掩码，真实实现必须加
    """
    B, H, S, D = q.shape
    scale = 1.0 / math.sqrt(D)
    n_blocks = S // block_size

    # ---------- 分支 1: 压缩块注意力（粗粒度全局）----------
    o_cmp = sdpa(q, compress_blocks(k, block_size), compress_blocks(v, block_size), scale)

    # ---------- 分支 2: top-n 块选择（中粒度重要区域）----------
    k_cmp = compress_blocks(k, block_size)                  # 用压缩表示来粗选
```

```
    scores = q @ k_cmp.transpose(-1, -2) * scale           # [B, H, S, n_blocks]
    top_idx = scores.topk(min(top_n, n_blocks), dim=-1).indices        # [B,H,S,top_n]

    # 把 K/V 重排成块，再按选中的块索引取回（块对齐，硬件友好）
    k_blk = k[:, :, :n_blocks * block_size].reshape(B, H, n_blocks, block_size, D)
    v_blk = v[:, :, :n_blocks * block_size].reshape(B, H, n_blocks, block_size, D)
    bidx = torch.arange(B, device=q.device).view(B, 1, 1, 1)
    hidx = torch.arange(H, device=q.device).view(1, H, 1, 1)
    k_sel = k_blk[bidx, hidx, top_idx]                     # [B,H,S,top_n,l,D]
    v_sel = v_blk[bidx, hidx, top_idx]
    k_sel = k_sel.reshape(B, H, S, -1, D)                  # [B,H,S,top_n*l,D]
    v_sel = v_sel.reshape(B, H, S, -1, D)
    o_slc = sdpa(q.unsqueeze(3), k_sel, v_sel, scale).squeeze(3)

    # ---------- 分支 3: 滑动窗口（局部精确）----------
    win = min(window, S)
    o_win = sdpa(q, k[:, :, -win:, :], v[:, :, -win:, :], scale)

    # ---------- 门控加权求和 ----------
    g = torch.sigmoid(q.mean(dim=(1, 3), keepdim=True)).expand(-1, 1, -1, 3)
    w_cmp, w_slc, w_win = g[..., 0:1], g[..., 1:2], g[..., 2:3]
    return w_cmp * o_cmp + w_slc * o_slc + w_win * o_win


class LightningIndexer(nn.Module):
    """DSA 的 lightning indexer: 轻量投影 + ReLU 打分 + top-k"""

    def __init__(self, n_heads, d_model, d_index=16):
        super().__init__()
        self.wq = nn.Linear(d_model, n_heads * d_index, bias=False)
        self.wk = nn.Linear(d_model, d_index, bias=False)
        self.w_scale = nn.Linear(d_model, n_heads, bias=False)    # 每个 head 的标量权重
        self.n_heads, self.d_index = n_heads, d_index

    def forward(self, h, top_k=8):
        """h: [B, S, D] 隐状态；返回每个 query 选中的 token 索引 [B, S, top_k]"""
        B, S, _ = h.shape
        q = self.wq(h).view(B, S, self.n_heads, self.d_index).transpose(1, 2)  # [B,H,S,d]
        k = self.wk(h)                                                         # [B,S,d]
        w = self.w_scale(h).transpose(1, 2)                                    # [B,H,S]

        # 每个 indexer head 打分后 ReLU（可跑 FP8），再用标量权重加权求和
        dot = q @ k.transpose(-1, -2)                                          # [B,H,S,S]
        score = (w * torch.relu(dot)).sum(dim=1)                               # [B,S,S]

        # 因果：只看过去
        causal = torch.tril(torch.ones(S, S, dtype=torch.bool, device=h.device))
        score = score.masked_fill(~causal, float("-inf"))
        return score.topk(min(top_k, S), dim=-1).indices                       # [B,S,top_k]


if __name__ == "__main__":
    torch.manual_seed(0)
    B, H, S, D = 1, 4, 64, 16
    q, k, v = (torch.randn(B, H, S, D) for _ in range(3))

    o = nsa_attention(q, k, v, block_size=8, top_n=2, window=16)
    print("NSA 输出:", tuple(o.shape), "(与输入同形)")
```

```
    indexer = LightningIndexer(H, D, d_index=16)
    idx = indexer(torch.randn(B, S, D), top_k=8)
    print("indexer 选中索引:", tuple(idx.shape))
    print("每个 query 只算 8 个 token → O(S·k) 而非 O(S²)")
```

🎯 面试要点

- **一句话**: 长上下文的注意力必须稀疏 —— NSA 靠三分支 + 门控，DSA 靠极轻的 indexer 选 top-k
- **三分支各解决什么**: 压缩块给粗粒度全局视野，top-n 选择聚焦重要中程区域，滑窗保证局部精确 —— 缺一个就有明显退化
- **为什么需要门控**: 三条分支的相对重要性随层数、位置、任务变化，硬相加不行，必须有可学习的权重
- **块级 vs token 级选择**: 块级选择（NSA）对硬件友好；DSA 的 token 级 top-k 靠极轻的 indexer 把选择成本压到可忽略
- **indexer 为什么能用 ReLU**: softmax 需要全序列归一化，ReLU 不需要，因此可以分块算、能跑 FP8 —— 这是「lightning」的工程前提
- **indexer 的维度要小**: `d^I` 刻意做得很小（几十维量级），选择成本才可以忽略
- **复杂度**: `O(S²)` → `O(S·k)`。`k=2048`、`S=128K` 时计算量约降两个数量级
- **训练不是一步到位**: DSA 先冻结主模型、只训 indexer 对齐分布（warm-up），再放开全部参数做稀疏训练
- **因果性**: 只能从当前 token 往前的历史里选，未来的块必须屏蔽
- **面试常问**: 稀疏注意力怎么保证不丢长程信息？为什么用块级别选择而不是 token 级别？

📉 损失函数 Loss

4 题

交叉熵 / 语言模型损失

Cross Entropy / LM Loss

下一个 token 预测，LLM 训练基础

📌 原理与思想

核心概念

语言模型的核心训练目标：给定前文预测下一个 token。通过 shift 操作将 logits 和 labels 对齐，计算交叉熵。交叉熵衡量预测分布与真实分布的差异，梯度计算简洁（softmax + CE 的梯度 = y - one_hot），是 LLM 训练的基础。

核心思想

用前面的 token 预测下一个 token。通过 shift 操作：logits 去尾（去掉最后一个位置的预测），labels 去头（去掉第一个位置的目标），使 logits[:, :-1] 预测 labels[:, 1:]。

算法步骤

1. Shift logits：shift_logits = logits[:, :-1, :]
2. Shift labels：shift_labels = labels[:, 1:]
3. 展平：flattened_logits = shift_logits.view(-1, V)
4. 展平：flattened_labels = shift_labels.view(-1)
5. 计算交叉熵：loss = CE(flattened_logits, flattened_labels)

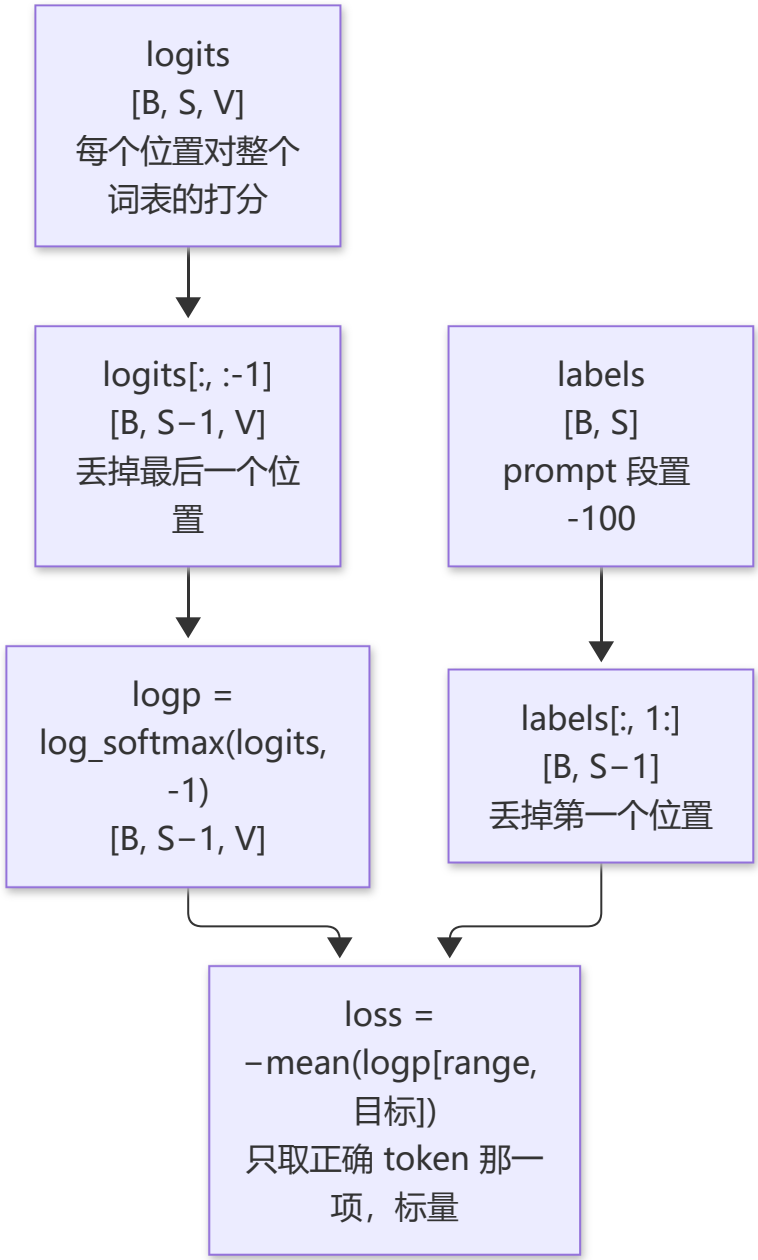
📐 核心公式

$$\mathcal{L}_{\text{CE}} = -\frac{1}{|x|}\sum_{t=1}^{|x|}\log P_\theta(x_t \mid x_{<t})$$

$$\text{CE}(\text{shift}(\text{logits}),\ \text{shift}(\text{labels}))\,, \qquad \text{logits}[:, :-1] \text{ predicts } \text{labels}[:, 1:]$$

$$\begin{aligned} \text{Pretrain}: &\quad \text{all tokens counted} \\ \text{SFT}: &\quad \text{label}_{\text{prompt}} = -100, \qquad \text{ignore_index} = -100 \end{aligned}$$

📊 张量流程图



💻 代码实现

```
import torch
import torch.nn.functional as F

def lm_loss(logits, labels, ignore_index=-100):
    """
    语言模型损失（下一个 token 预测）

    logits: [B, S, vocab_size]
    labels: [B, S]
    """
    # Shift: 用前面的 token 预测下一个
    shift_logits = logits[:, :-1, :].contiguous()   # [B, S-1, V]
```

```
    shift_labels = labels[:, 1:].contiguous()        # [B, S-1]

    return F.cross_entropy(
        shift_logits.view(-1, shift_logits.size(-1)),
        shift_labels.view(-1),
        ignore_index=ignore_index
    )

# 测试
if __name__ == "__main__":
    B, S, V = 2, 10, 32000
    logits = torch.randn(B, S, V)
    labels = torch.randint(0, V, (B, S))

    loss = lm_loss(logits, labels)
    print(f"Loss: {loss.item():.4f}")

# SFT Loss: 只计算 response 部分
def sft_loss(logits, labels, prompt_lengths):
    """
    SFT 损失：prompt 部分设为 -100
    """
    masked_labels = labels.clone()
    for i, plen in enumerate(prompt_lengths):
        masked_labels[i, :plen] = -100
    return lm_loss(logits, masked_labels)
```

> 🎯 **面试要点**

- **Shift 操作是核心**: logits 去尾，labels 去头
- **ignore_index=-100** 的位置不参与梯度
- **Pretrain**: 所有 token 参与 loss
- **SFT**: prompt 部分 label 设为 -100，只算 response
- **面试常问**: Pretrain Loss 和 SFT Loss 的区别?
- **梯度形式**: ∂L/∂logits = softmax(logits) − one_hot(目标)，预测越离谱梯度越大

DPO 损失

Direct Preference Optimization

无需奖励模型，直接优化偏好数据

📌 原理与思想

核心概念

将奖励函数参数化为策略与参考策略的对数比率，直接在偏好数据上优化。增加 chosen 概率，降低 rejected 概率。相比 PPO 不需要 reward model 和 critic，训练更简单稳定，效果相当甚至更好。

核心思想

利用 RL 的对偶性，将奖励函数隐式表示为 r(x,y) = β · log(πθ(y|x)/πref(y|x))。这样策略优化目标可以直接用策略的对数概率差来表示，无需显式奖励模型。

算法步骤

1. 收集偏好数据：(prompt, chosen_response, rejected_response)
2. 计算策略与参考策略的对数概率差
3. 构造损失函数：增加 chosen 概率，降低 rejected 概率
4. 直接用梯度下降优化策略模型

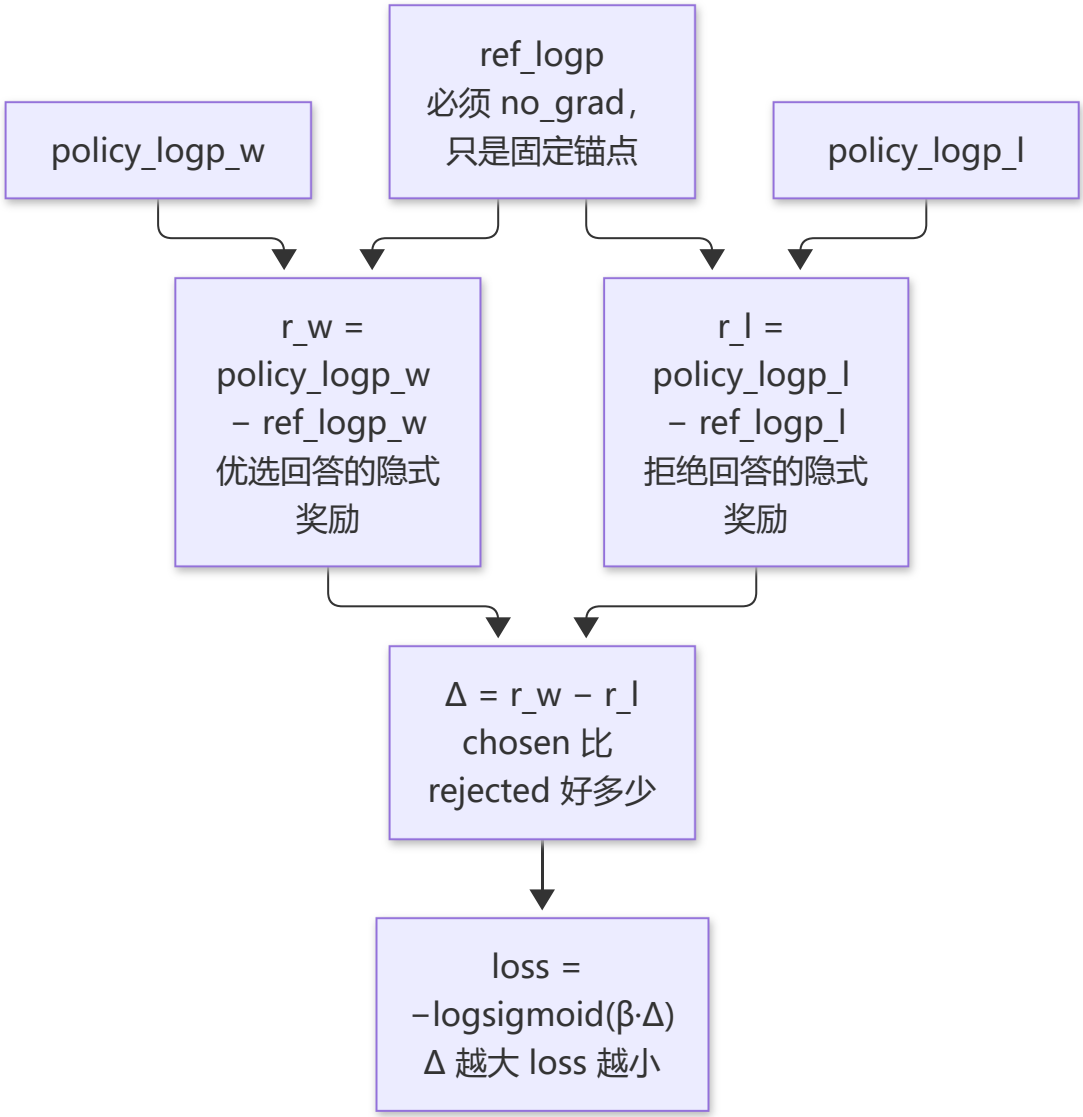
📐 核心公式

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta\left(\log\frac{\pi_\theta(y_w\mid x)}{\pi_{\text{ref}}(y_w\mid x)} - \log\frac{\pi_\theta(y_l\mid x)}{\pi_{\text{ref}}(y_l\mid x)}\right)\right)\right]$$

$$\text{logits} = \left(\log p_\theta^w - \log p_{\text{ref}}^w\right) - \left(\log p_\theta^l - \log p_{\text{ref}}^l\right), \qquad \mathcal{L} = -\log\sigma(\beta\cdot\text{logits})$$

$$y_w:\ \text{chosen}, \qquad y_l:\ \text{rejected}, \qquad \beta\in[0.1,\ 0.5]$$

📊 张量流程图



💻 代码实现

```python
import torch
import torch.nn.functional as F

def dpo_loss(policy_chosen_logps, policy_rejected_logps,
             ref_chosen_logps, ref_rejected_logps, beta=0.1):
    """
    DPO 损失

    *_logps: [batch_size] 每个样本的对数概率之和
    """
    # 隐式奖励 = log(π_θ / π_ref)
    chosen_ratio = policy_chosen_logps - ref_chosen_logps
    rejected_ratio = policy_rejected_logps - ref_rejected_logps

    # 奖励差
    logits = chosen_ratio - rejected_ratio
```

```
    # DPO 损失
    return -F.logsigmoid(beta * logits).mean()

# 完整训练流程
def compute_logps(model, input_ids, labels):
    """计算模型对 labels 的对数概率之和"""
    logits = model(input_ids).logits
    log_probs = F.log_softmax(logits[:, :-1], dim=-1)
    labels = labels[:, 1:]
    per_token_logps = log_probs.gather(-1, labels.unsqueeze(-1)).squeeze(-1)
    return per_token_logps.sum(dim=-1)

# 测试
if __name__ == "__main__":
    B = 4
    policy_chosen = torch.randn(B)
    policy_rejected = torch.randn(B)
    ref_chosen = torch.randn(B)
    ref_rejected = torch.randn(B)

    loss = dpo_loss(policy_chosen, policy_rejected,
                    ref_chosen, ref_rejected, beta=0.1)
    print(f"DPO Loss: {loss.item():.4f}")
```

🎯 面试要点

- **核心**: 奖励 = log(π_θ / π_ref)，无需显式奖励模型
- **chosen 概率↑ + rejected 概率↓** → 对齐人类偏好
- **β 控制偏离参考模型程度**，通常 0.1-0.5
- **比 PPO 简单很多**: 不需要 reward model 和 critic
- **变体**: IPO, KTO, ORPO, SimPO 等
- **面试常问**: DPO 和 PPO 的区别？为什么 DPO 更简单？
- **参考模型的 logp 必须 no_grad**: 它只是固定锚点，只有策略一侧回传梯度

GRPO 损失

Group Relative Policy Optimization

去掉 Critic，组内归一化优势，DeepSeek-R1

📌 原理与思想

核心概念

PPO 的简化版：对同一问题生成 G 个回答，用组内归一化的奖励作为优势，不需要 Critic 网络。相比 PPO 节省约 40% 训练显存，是 DeepSeek-R1 使用的强化学习算法。

核心思想

对同一问题生成 G 个回答，用组内归一化的奖励代替 Critic 网络的价值估计。优势函数 A_i = (r_i - mean(r)) / (std(r) + ε)，结合 PPO clip 机制和显式 KL 惩罚。

组内的优势必然有正有负，梯度才有明确方向；clip 限制单步更新幅度，显式 KL 惩罚则拉住参考策略、防止跑偏。对比 PPO：PPO 要先训一个 Critic 估计 V(s)，再用 A = r + γV(s) − V(s)，GRPO 省掉的正是这一步。

算法步骤

1. 对同一问题 q 生成 G 个回答：o_1, o_2, ..., o_G
2. 计算每个回答的奖励：r_1, r_2, ..., r_G
3. 组内归一化优势：A_i = (r_i - mean) / (std + ε)
4. 计算 PPO clip 损失
5. 添加显式 KL 惩罚：loss += β · KL(π||π_ref)

核心公式

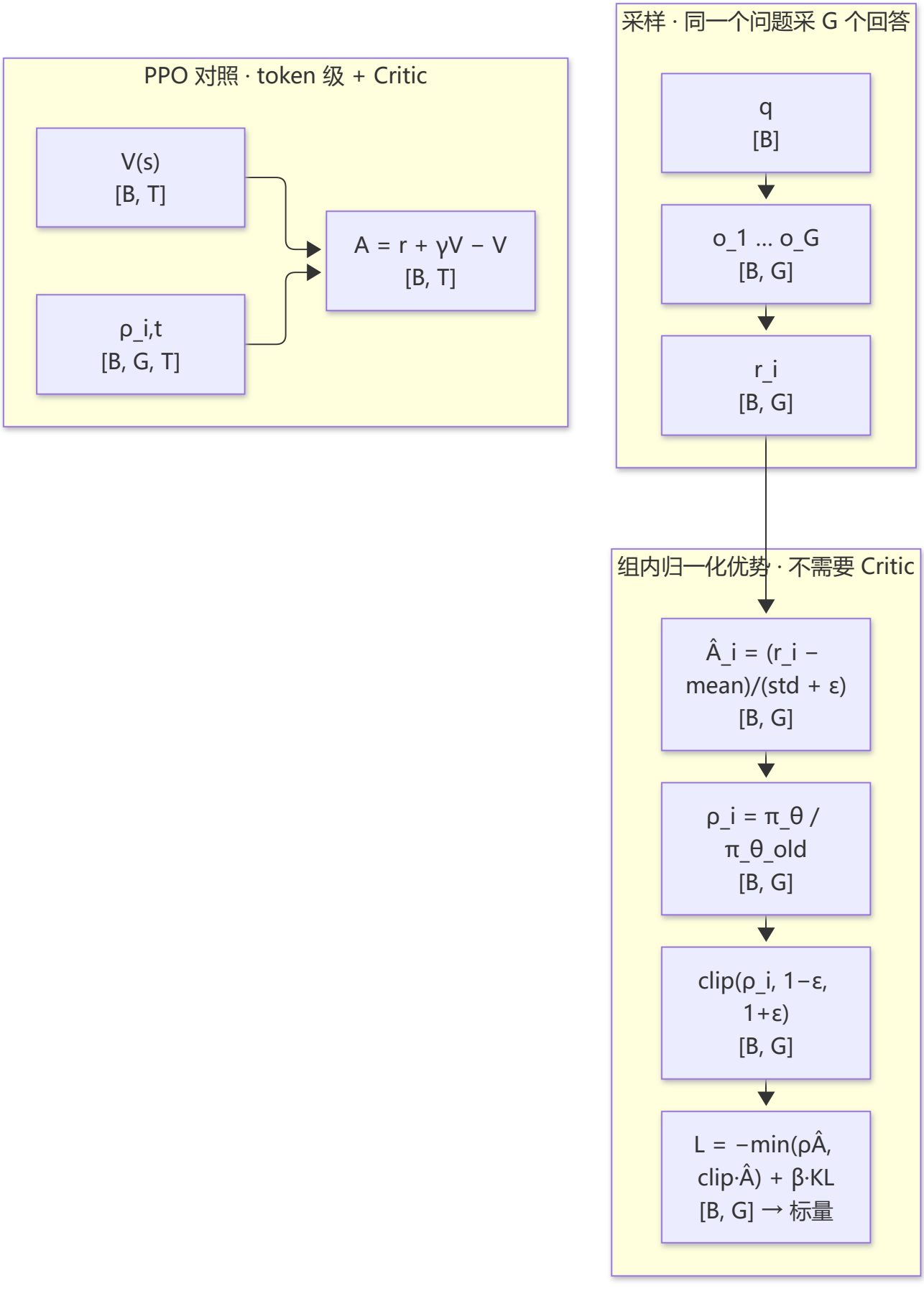
$$\hat{A}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r}) + \epsilon}, \qquad \{o_1, \ldots, o_G\} \sim \pi_{\theta_{\text{old}}}(\cdot \mid q)$$

$$\mathcal{L}(\theta) = -\mathbb{E}\Big[\min\Big(\rho_i \hat{A}_i,\ \text{clip}(\rho_i, 1-\epsilon, 1+\epsilon)\,\hat{A}_i\Big)\Big] + \beta\,\text{KL}(\pi_\theta \,\|\, \pi_{\text{ref}})$$

$$\text{GRPO (sequence-level):} \quad \rho_i = \frac{\pi_\theta(o_i \mid q)}{\pi_{\theta_{\text{old}}}(o_i \mid q)}$$

$$\text{PPO (token-level):} \quad \rho_{i,t} = \frac{\pi_\theta(o_{i,t} \mid x, y_{<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid x, y_{<t})}$$

张量流程图


采样 · 同一个问题采 G 个回答
q
[B]
o_1 … o_G
[B, G]
r_i
[B, G]
PPO 对照 · token 级 + Critic
V(s)
[B, T]
ρ_i,t
[B, G, T]
A = r + γV − V
[B, T]
组内归一化优势 · 不需要 Critic
Â_i = (r_i − mean)/(std + ε)
[B, G]
ρ_i = π_θ / π_θ_old
[B, G]
clip(ρ_i, 1−ε, 1+ε)
[B, G]
L = −min(ρÂ, clip·Â) + β·KL
[B, G] → 标量

💻 代码实现

```python
import torch

def grpo_advantages(rewards):
    """组内归一化优势 (不需要 Critic)"""
    # rewards: [batch, group_size]
    mean = rewards.mean(dim=-1, keepdim=True)
    std = rewards.std(dim=-1, keepdim=True)
    return (rewards - mean) / (std + 1e-8)

def grpo_loss(old_logp, new_logp, advantages, eps=0.2, beta=0.01, ref_kl=None):
    """
    GRPO 损失 = PPO clip loss + β * KL penalty
    """
    ratio = torch.exp(new_logp - old_logp)
    clipped = torch.clamp(ratio, 1-eps, 1+eps)
    loss = -torch.min(ratio * advantages, clipped * advantages)
    if ref_kl is not None:
        loss = loss + beta * ref_kl       # 显式 KL 惩罚
    return loss.mean()

def kl_penalty(logp, ref_logp):
    """Schulman KL 估计器 (低方差)"""
    ratio = torch.exp(ref_logp - logp)
    return (ratio - (ref_logp - logp) - 1).mean()

# 完整 GRPO 训练流程
class GRPOTrainer:
    def __init__(self, policy_model, ref_model, reward_model, group_size=8):
        self.policy = policy_model
        self.ref = ref_model
        self.reward = reward_model
        self.group_size = group_size

    def train_step(self, prompts):
        # 1. 对每个 prompt 生成 group_size 个回答
        responses = []
        for _ in range(self.group_size):
            responses.append(self.policy.generate(prompts))

        # 2. 计算每个回答的 reward
        rewards = torch.stack([
            self.reward(prompts, resp) for resp in responses
        ], dim=1)  # [batch, group_size]

        # 3. 组内归一化优势
        advantages = grpo_advantages(rewards)

        # 4. GRPO 更新
        for i in range(self.group_size):
            old_logp = self.policy.get_logprobs(prompts, responses[i])
            new_logp = self.policy.get_logprobs(prompts, responses[i])
            ref_logp = self.ref.get_logprobs(prompts, responses[i])

            kl = kl_penalty(new_logp, ref_logp)
            loss = grpo_loss(old_logp, new_logp, advantages[:, i],
```

```
                            ref_kl=kl)
            loss.backward()

# 测试
if __name__ == "__main__":
    # 组内归一化
    rewards = torch.tensor([[1.0, 2.0, 3.0, 0.5],
                            [2.0, 2.5, 1.5, 3.0]])
    advantages = grpo_advantages(rewards)
    print("Advantages:", advantages)
    # 每行均值为 0，标准差为 1
```

🎯 面试要点

- **核心**: 组内归一化代替 Critic 网络
- **对同一问题生成 G 个回答**，计算组内相对优势
- **不需要 Value Head**，节省约 40% 训练显存
- **显式 KL 惩罚**防止偏离参考策略太远
- **DeepSeek-R1 使用 GRPO** 进行强化学习
- **面试常问**: GRPO 和 PPO 的区别？为什么不需要 Critic?

在线策略蒸馏

On-Policy Distillation

学生自己生成轨迹，教师逐 token 给稠密监督

📌 原理与思想

核心概念

让学生模型自己采样生成轨迹，再让教师模型在学生实际走过的每个 token 上给出完整分布作为监督信号。相比用教师生成的静态数据做离线蒸馏，on-policy 训练消除了训练与推理的分布不匹配（exposure bias）；相比只有稀疏结果奖励的 RL，教师的逐 token 分布本身就是一个稠密奖励，不需要额外的 reward model。代价是每个 batch 都要现场采样，且必须有一个更强的教师模型可用。

核心思想

让学生自己采样，教师在被采样出来的轨迹上逐 token 给出完整分布 —— 监督落在学生真正会走的路径上，训练分布与推理分布从此一致，exposure bias 被从根上消掉。教师的分布在每个位置都是一个几万维的概率向量，本身就是一个稠密奖励，不需要 reward model。方向用反向 KL：mode-seeking 让学生只去对齐教师的高概率模式，与 Hinton 蒸馏正好相反。

算法步骤

1. 学生前向并采样得到 `y_S`，它带着学生自己的错误 —— 这正是要让它学会纠正的地方。
2. 教师在同一批 `prompt + y_S` 上做一次前向，得到 `teacher_logits: [B, T, V]`，必须 `detach` / `no_grad`。
3. 逐 token 算散度 `D(p_T ‖ p_S)`，得到 `per_token_loss: [B, T]`，再按有效 token 数求平均。
4. 用广义 JSD 插值统一方向：`β = 0` 是前向 KL（mode-covering）、`β = 1` 是反向 KL（mode-seeking）、`β = 0.5` 就是标准 JSD。

📐 核心公式

$$\mathcal{L}_{\text{OPD}} = \mathbb{E}_{y \sim \pi_S(\cdot|x)} \left[\frac{1}{|y|} \sum_t D(p_T(\cdot \mid x, y_{<t}) \,\|\, p_S(\cdot \mid x, y_{<t})) \right]$$

$$M = \beta\, p_T + (1-\beta)\, p_S$$

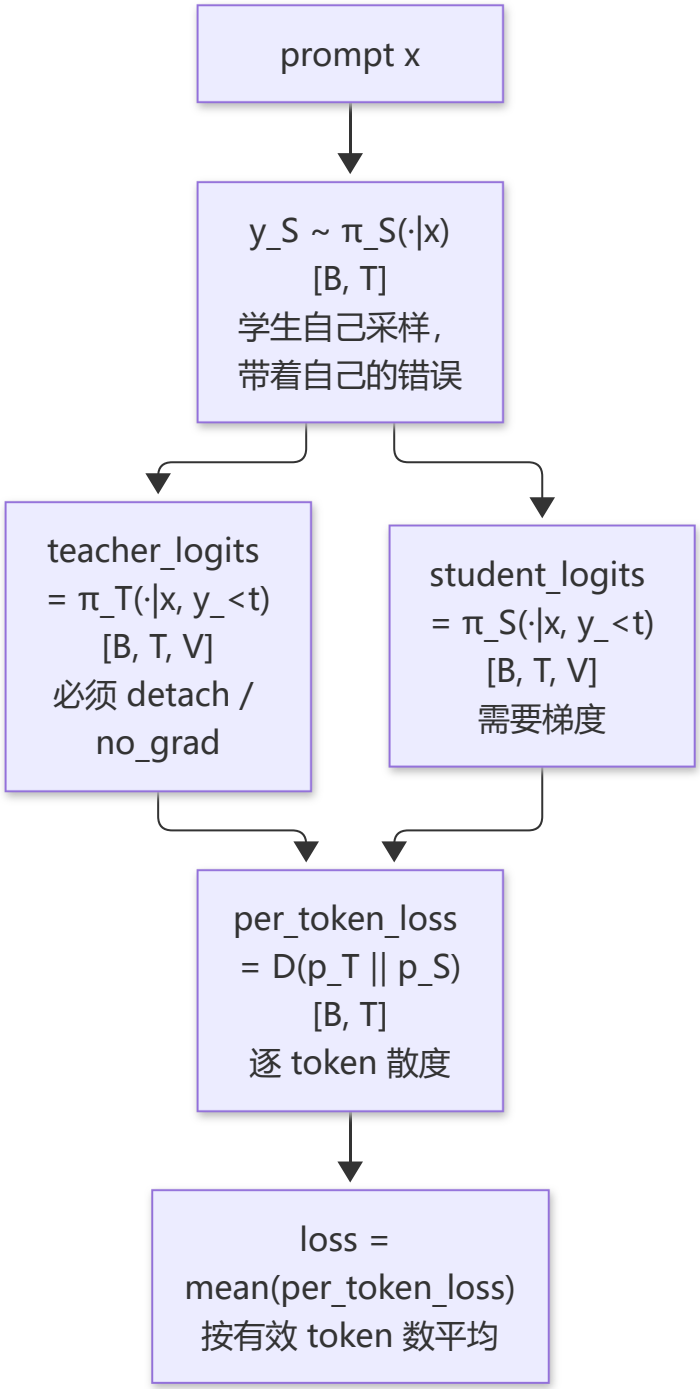
$$D_{\text{GJS}}(\beta) = (1-\beta)\text{KL}(p_T \| M) + \beta \text{KL}(p_S \| M), \qquad \beta \in [0,1]$$

$$\beta = 0 \Rightarrow \text{KL}(p_T \| p_S) \qquad \text{forward KL (mode-covering)}$$

$$\beta = 1 \Rightarrow \text{KL}(p_S \| p_T) \qquad \text{reverse KL (mode-seeking)}$$

$$\beta = 0.5 \Rightarrow \text{standard JSD}$$

📊 张量流程图



💻 代码实现

```
import torch
import torch.nn.functional as F

def generalized_jsd(student_logits, teacher_logits, beta=1.0, temperature=1.0):
    """
    广义 JSD 插值
    beta = 0   → 前向 KL  KL(p_T ‖ p_S)   (mode-covering)
    beta = 1   → 反向 KL  KL(p_S ‖ p_T)   (mode-seeking)
```

```
    beta = 0.5 → 标准 JSD
    student_logits, teacher_logits: [B, T, V]，未归一化的 logits
    返回逐 token 散度 [B, T]
    """
    # 1. 温度缩放 + log_softmax（数值稳定）
    s_logp = F.log_softmax(student_logits / temperature, dim=-1)
    t_logp = F.log_softmax(teacher_logits / temperature, dim=-1)
    s_p = s_logp.exp()
    t_p = t_logp.exp()

    # 2. 混合分布 M = beta * p_T + (1 - beta) * p_S
    #    clamp_min 只是防止 log(0)，V 个 token 上概率不会真的到 0
    m_logp = (beta * t_p + (1 - beta) * s_p).clamp_min(1e-8).log()

    # 3. KL(p_T ‖ M) 与 KL(p_S ‖ M)，在词表维度求和
    kl_t = (t_p * (t_logp - m_logp)).sum(dim=-1)     # [B, T]
    kl_s = (s_p * (s_logp - m_logp)).sum(dim=-1)     # [B, T]

    return (1 - beta) * kl_t + beta * kl_s           # [B, T]


def opd_loss(student_logits, teacher_logits, loss_mask=None,
             beta=1.0, temperature=1.0, top_k=0):
    """
    在线策略蒸馏损失
    loss_mask: [B, T]，1 = 学生在该位置真实采样出的 token
    top_k:     >0 时只保留教师概率最高的 k 个 token
               （黑盒教师只吐 top-k logits 时用，其余位置截断）
    """
    # 教师永远不参与反传，否则「教师」会被同一步更新带跑
    teacher_logits = teacher_logits.detach()

    if top_k > 0:
        # 教师只给 top-k：其余位置截断到一个很大的负数
        # 用 -1e9 而不是 -inf，避免 0 * inf 产生 nan
        kth = teacher_logits.topk(top_k, dim=-1).values[..., -1:]
        teacher_logits = teacher_logits.masked_fill(teacher_logits < kth, -1e9)

    per_token = generalized_jsd(student_logits, teacher_logits, beta, temperature)

    if loss_mask is None:
        return per_token.mean()
    mask = loss_mask.float()
    return (per_token * mask).sum() / mask.sum().clamp_min(1.0)


if __name__ == "__main__":
    torch.manual_seed(0)
    B, T, V = 2, 4, 8
    student = torch.randn(B, T, V)
    teacher = torch.randn(B, T, V)

    # 端点自检：beta=0 应等于前向 KL，beta=1 应等于反向 KL
    s_logp = F.log_softmax(student, dim=-1)
    t_logp = F.log_softmax(teacher, dim=-1)
    # F.kl_div(input, target, log_target=True) = KL(target ‖ input)
    fwd = F.kl_div(s_logp, t_logp, log_target=True, reduction='none').sum(-1)
    rev = F.kl_div(t_logp, s_logp, log_target=True, reduction='none').sum(-1)
```

```
print("beta=0  :", generalized_jsd(student, teacher, beta=0.0).mean().item())
print("  前向KL:", fwd.mean().item())
print("beta=1  :", generalized_jsd(student, teacher, beta=1.0).mean().item())
print("  反向KL:", rev.mean().item())
print("beta=0.5:", generalized_jsd(student, teacher, beta=0.5).mean().item(), "(标准 JSD)")

# top-k 教师 + mask
mask = torch.ones(B, T)
mask[:, -1] = 0
print("top_k=3:", opd_loss(student, teacher, mask, beta=1.0, top_k=3).item())
```

🎯 面试要点

- **为什么要 on-policy**: 消除 exposure bias。离线蒸馏的误差累积是 $O(T^2)$，on-policy 降到 $O(T)$
- **为什么常选反向 KL**: 反向 KL 是 mode-seeking，学生只去匹配教师的高概率模式；前向 KL 是 mode-covering，会给教师几乎不给概率的区域也分配质量
- **KL 方向决定行为**: 前向 KL 覆盖所有模式但分布更散（易产生幻觉区域），反向 KL 更尖锐但可能丢多样性
- **与 RL 的区别**: 不需要 reward model，教师逐 token 的分布就是稠密奖励；同等效果下比 RL 便宜得多
- **教师的 logits 必须 detach**: 教师是「固定靶」，只有学生一侧回传梯度
- **黑盒教师**: 拿不到全词表 logits 时，用 top-k logits 截断近似，其余位置填一个很大的负数（不要填 `-inf`，会在 `0 * inf` 处产生 nan）
- **广义 JSD 一个公式统一了前后向 KL**: `β` 从 0 滑到 1，就是从 mode-covering 滑到 mode-seeking，代码里用端点数值断言自检
- **面试常问**: on-policy 蒸馏和 RLHF 是什么关系？为什么 OPD 不需要 reward model?